Andrés Vélez Rendón · Santiago Vélez Blandón · Eduardo Andrés Piñeros Manjarrés

1. Qué se construyó y cómo se ejecutó

Se completaron los TODO de `ingest.py`, `producer.py`, los cinco jobs (`bronze`, `silver`, `gold`, `audit`, `stream`), `athena.sql` y `serverless.yml`. Todo se desarrolló y validó primero en local (Docker + Spark 3.5 + Delta 3.2 + Kafka) y luego se desplegó en AWS Academy Learner Lab (S3, Glue 5.0, Kinesis, Athena). El diagrama de la arquitectura está en `informe/diagrama/` (`.drawio`, `.pdf`, `.png`).

Prueba	Local	AWS
`pytest`	15/15 en verde	—
Lote 1 (2026-09-29)	validaciones ✓; 2.ª ejecución: 0 insertadas, 0 actualizadas	4 jobs de Glue *Succeeded*
Lote 2 (2026-09-30)	✓; 1.ª: 1 883 insertadas, 276 actualizadas; 2.ª: 0 y 0	4 jobs *Succeeded*
Stream *availableNow*	12 310 eventos; 2.ª ejecución: 0 filas leídas, tablas sin cambios	—
Stream en vivo	4 858 filas en 180 s, validaciones ✓	`paytrack-stream` *Succeeded* (9 792 eventos, 109 ventanas, 89 alertas)
Athena	6 consultas probadas con Spark SQL	6 consultas ejecutadas (§4)

Incidencias. (i) Learner Lab no permite crear roles de IAM: `serverless.yml` usa el `LabRole` existente. (ii) El conector de Kinesis de Glue 5.0 entrega la hora de llegada en la columna `$add$record_timestamp$_temporary$` y no en `__src_timestamp`; `read_events` (en `common.py`, archivo dado) fallaba con `KeyError`. Se diagnosticó imprimiendo `df.dtypes` en una ejecución corta y se corrigió aceptando ambos nombres (dos corridas *Failed* previas en el historial de Glue).

2. Parte A · Batch

1. Bronze. Lee todo como texto (`primitivesAsString`) porque el tipado es responsabilidad de Silver: un lote con un tipo distinto (p. ej. montos como `"40380.23"`, 600 de 30 600 filas del lote 1) no rompe la tabla ni pierde el dato original. `replaceWhere` sobre `ingest_date` reemplaza solo la partición del lote; con `append`, re-ejecutar duplicaría el lote (se tendrían 61 200 filas de transacciones en lugar de 30 600) y la validación "sin duplicar" fallaría (no lo ejecutamos así: es el razonamiento, no una medición). En nuestra prueba, la segunda ejecución del lote 1 dejó 30 600, 125, 500 y 8 filas, idéntico a la primera.

2. Silver. Cuarentena del lote 1: `invalid_amount` 1 180 y `missing_customer` 589; del lote 2: 77 y 40; ninguna por `invalid_status`. Se deduplica *antes* de aplicar las reglas para que cada `transaction_id` se evalúe una sola vez, con su versión más reciente (`ingested_at`): si se evaluara antes, un duplicado malo y su copia buena podrían caer en cuarentena y en Silver a la vez, y los conteos válidas + cuarentena no cuadrarían con las transacciones distintas.

3. MERGE. Actualizar solo si el `status` cambió evita reescribir filas idénticas y hace el lote idempotente: la 2.ª ejecución del lote 2 reportó 0 insertadas y 0 actualizadas. Sin esa condición, `numTargetRowsUpdated` sería igual al número de transacciones del lote ya existentes en Silver y la auditoría mostraría "cambios" que no lo son (`old_status = new_status`, que la validación rechaza).

4. Time travel. La auditoría encontró **276 contracargos** (`approved→chargeback`) por **COP 20 712 854,10** (en Athena, por categoría: retail 72, grocery 43, delivery 40, streaming 39, telco 31, travel 27, transport 24). Tras un `VACUUM` con retención menor a la antigüedad de la versión, Delta borra los archivos de datos que solo esa versión referencia y `versionAsOf` dejaría de funcionar: la auditoría no podría reconstruir el estado anterior (por defecto la retención es de 7 días).

5. PII. Con tipo 1 se pierde el historial: no se puede saber en qué ciudad o segmento estaba un cliente cuando compró (análisis de migración entre segmentos, ventas por ciudad en el momento de la compra). Se resolvería con SCD tipo 2 como en el Lab 1: `valid_from`, `valid_to`, `is_current`, con un `MERGE` que cierra la fila vigente e inserta la nueva.

6. Consumo. Consulta 1 (Gold): 2,89 KB escaneados; consulta 2 (Silver, misma pregunta): 93,94 KB, unas 32 veces más, porque Silver es a nivel de transacción y Gold ya trae el agregado diario. Gold existe para dar respuestas rápidas y baratas a preguntas frecuentes; lo que se pierde es flexibilidad (solo responde lo que se pre-agregó) y frescura: Gold se recalcula con el job, y por eso los montos de las consultas 1 y 2 difieren (Gold no se recalculó tras los contracargos que llegaron por stream).

3. Parte B · Streaming

7. Garantías de entrega. El productor Kafka (`acks=all`, `enable.idempotence=true`) no pierde ni duplica por reintentos propios, pero la *fuente* reenvía eventos a propósito (3 %), así que el sistema es **at-least-once**. Con Kinesis, `put_with_retries` reintenta solo los fallidos con *backoff*; un reintento tras un fallo ambiguo puede duplicar, también at-least-once. El efecto "exactamente una vez" sale de la combinación: el *checkpoint* recuerda los offsets leídos; la deduplicación por `event_id`+`event_time`

con watermark descarta reenvíos; las escrituras Delta son atómicas y `MERGE` por `transaction_id` es idempotente. Evidencia: la 2.ª ejecución leyó 0 filas y las tablas siguieron en 41 796 transacciones, 103 ventanas y 110 alertas.

8. Watermark. Filas descartadas por watermark en `metrics`: en modo *availableNow* (`-fast`) **3 de 12 289** (0,02 %); en vivo **46 de 2 459** (1,9 %) y 147 de 6 098 (2,4 %) en la prueba larga; en AWS 56 de 9 792 (0,6 %, el job leyó parte del histórico retenido en Kinesis). El productor envía 3 % de eventos tardíos con retraso de 30 a 360 s y el watermark es de 120 s: en vivo el watermark sigue la cabeza del stream en micro-lotes de 10 s y se descarta todo lo que llega más de 120 s tarde ($\approx (360-120)/(360-30) = 73\,\%$ de los tardíos, ≈2,2 % de los eventos, justo lo medido). En *availableNow* los micro-lotes son grandes y el watermark avanza solo al cerrar cada uno, así que casi todos los tardíos caen dentro del mismo lote y se toleran. `silver` no usa watermark porque su `MERGE` no guarda estado que acotar y un pago es un hecho que no puede perderse; con watermark se descartaría ≈2 % de los pagos tardíos y Silver quedaría incompleto (la validación "llegaron todas las transacciones del stream, también las tardías" fallaría).

9. Latencia. Medida en local con tráfico continuo (5 min a 20 ev/s, 31 alertas verdaderas): entre el **último pago** de la ráfaga y el momento en que la alerta queda escrita en `gold.fraud_alerts` pasan **133,5 s como mínimo, 155,8 s de mediana y 171,8 s como máximo**. En modo *append*, una ventana solo se emite cuando el watermark (máximo `event_time` − 2 min) supera su `window_end`: son 120 s de watermark, hasta 30 s hasta el cierre de la ventana deslizante tras el último pago y hasta 10 s del disparo del micro-lote. Además, requiere que siga llegando tráfico para que el watermark avance: en una prueba de 2 min sin tráfico posterior no se emitió ninguna alerta. Se bajaría reduciendo el watermark (p. ej. 20–30 s, latencia ≈40–70 s) o usando modo *update*/`foreachBatch` para alertar con conteos parciales, a costa de descartar más eventos tardíos (con 30 s se perderían casi todos) o de emitir alertas provisionales repetidas.

10. Precisión. Precisión (aciertos/alertas): **20 ev/s: 127/163 = 77,9 %** en local y **89 alertas con 68 aciertos = 76,4 %** en AWS (consulta 5); **50 ev/s: 134/1 295 = 10,4 %** (local, modo `-fast`). Con 500 clientes, el número esperado de pagos legítimos de un cliente en una ventana de 30 s es $\lambda = \text{tasa} \times 30/500$: 1,2 a 20 ev/s y 3,0 a 50 ev/s; $P(N \geq 6)$ pasa de ≈0,1 % a ≈8,4 %, así que muchos clientes normales cruzan el umbral y las alertas falsas se multiplican mientras las verdaderas (las ráfagas) apenas crecen. Se mejoraría con un umbral que dependa de la tasa o del comportamiento histórico del cliente (media + k desviaciones), y con señales adicionales (rechazos, comercios distintos, montos, `risk_score`).

11. Estado final. Si se quitara `t.status <> 'chargeback'`, un *reenvío del pago* (o un pago tardío) de una transacción cuyo contracargo ya se integró, llegado en un micro-lote *posterior*, tendría `status = 'approved'` distinto al de la tabla y lo revertiría. `latest_per_transaction` solo mira un micro-lote: no ve el estado que ya está en Silver, así que no basta.

12. Partition key. Eventos por partición (60 s a 20 ev/s, 1 227 eventos, 3 particiones): `customer_id` 489/344/394; `transaction_id` 429/397/401; llave constante 0/1 227/0. La llave por cliente y por transacción reparten bien; la constante manda todo a una partición (un solo consumidor y, en Kinesis, un solo shard, con su límite de 1 000 registros/s o 1 MB/s ⇒ `ProvisionedThroughputExceeded`, para lo que sirven los reintentos). Con `transaction_id` *no cambiaría* el número de alertas de Spark: las ventanas por cliente se calculan tras un *shuffle* por `customer_id` usando `event_time`, no el orden del broker. La llave sirve para el orden y la colocalización *en el broker*: el contracargo lleva el mismo `transaction_id` y `customer_id` que su pago, así que con cualquiera de las dos llaves cae en la misma partición y se lee después del pago; la llave por cliente además mantiene en orden la historia completa de cada cliente (útil para otros consumidores), mientras que la constante no da paralelismo.

13. Batch y streaming en la misma tabla. Tras el streaming local, `DESCRIBE HISTORY` de `silver.transactions` mostró 23 versiones (0–22), de las cuales 18 son `MERGE` del stream; cada uno reescribió entre ≈16 000 y 41 000 filas (`numTargetRowsCopied`) para insertar o actualizar entre 77 y ≈3 900 (ejecución inicial de la prueba local). Incluso un `MERGE` sin cambios (2.ª ejecución del lote 1) copió 28 231 filas: Delta reescribe los archivos completos que toca. A 500 K pagos/día esa amplificación de escritura crece con el tamaño de la tabla; se mitigaría con micro-lotes más grandes, particionando/ordenando por `transaction_id` o por fecha para acotar los archivos tocados, *deletion vectors* y `OPTIMIZE` programado. Si el job batch y el stream escribieran a la vez, Delta usa concurrencia optimista: los `MERGE` que tocan los mismos archivos entran en conflicto (`ConcurrentModificationException`) y uno debe reintentar. Con la consulta 6, la **fuente de verdad es Silver**: los pagos del stream en Silver superan a los de las métricas en 333–390 por comercio, porque las métricas descartan eventos tardíos por watermark y excluyen las ventanas aún abiertas; la capa rápida es una aproximación de baja latencia.

14. Costos y operación. Precios de lista aproximados de us-east-1, 500 K transacciones/día: **Glue Streaming 24/7** con 2 DPU: 2 × 730 h × 0,44 ≈ USD 642; **Kinesis** 2 shards: 2 × 730 × 0,015 ≈ USD 22 (+ ≈ USD 0,2 de PUT); **Glue batch** (4 jobs diarios ≈ 0,4 DPU-h): ≈ USD 5–20; **S3** (decenas de GB con raw+Delta): ≈ USD 1–3; **Athena** (USD 5/TB, consultas de KB–MB): < USD 1. Total ≈ **USD 670–690/mes**, dominado por el job de streaming siempre encendido. Si no se necesitara 24/7, se programaría `availableNow` cada 15 min (unos 6,4 DPU-h/día ≈ USD 85/mes) o cada hora (≈ USD 20), o se usaría Firehose→S3. No se debe poner expiración de S3 sobre `lakehouse/`: borraría archivos de datos que el `_delta_log` sigue referenciando y corrompería las tablas; para limpiar versiones viejas se usa `VACUUM`, que respeta el log.

4. Athena (AWS)

N.º	Datos analizados	Resultado
1	2,89 KB	Top 5 desde Gold: Éxito Laureles COP 154,7 M (99,14 %), Claro Tienda, TuBodega.com, Rappi, Falabella
2	93,94 KB	Misma pregunta desde Silver: Éxito Laureles COP 154,4 M (98,74 %)
3	0,34 KB	Semana pico de fraude: 2026-09-07 en los tres segmentos (high_value 6, mass 10, premium 6)
4	289,13 KB	276 contracargos, COP 20,7 M (retail 72, grocery 43, delivery 40, . . .)
5	113,57 KB	89 alertas, 68 aciertos, **76,4 %** de precisión
6	203,20 KB	Pagos del stream: silver supera a la capa rápida en 333–390 por comercio (2026-11-03)

5. Experimentos

Exp. 1 · Watermark (descartadas por watermark en `metrics/alerts`):

Modo	Entorno	Filas leídas	Descartadas
availableNow (`-fast`, 20 ev/s)	local	12 289	3 (0,02 %)
En vivo (20 ev/s, 120 s)	local	2 459	46 (1,9 %)
En vivo (20 ev/s, 300 s)	local	6 098	147 (2,4 %)
En vivo (20 ev/s, 240 s) y backlog	AWS	9 792	56 (0,6 %)

Exp. 2 · Carga (precisión de las alertas, consulta 5 adaptada):

Tasa	Entorno	Alertas	Aciertos	Precisión
20 ev/s	AWS (en vivo)	89	68	76,4 %
20 ev/s	local (`-fast`, 600 s)	163	127	77,9 %
50 ev/s	local (`-fast`, 600 s)	1 295	134	10,4 %

Limitación: la prueba a 50 ev/s no se repitió en AWS (para no mantener Kinesis encendido) y se hizo en local con 600 s de tiempo de evento simulado sobre un lakehouse que ya traía el estado de checkpoints de pruebas previas, por lo que sus descartes por watermark (16 873) no son comparables con el resto y su conteo de alertas es un mínimo.

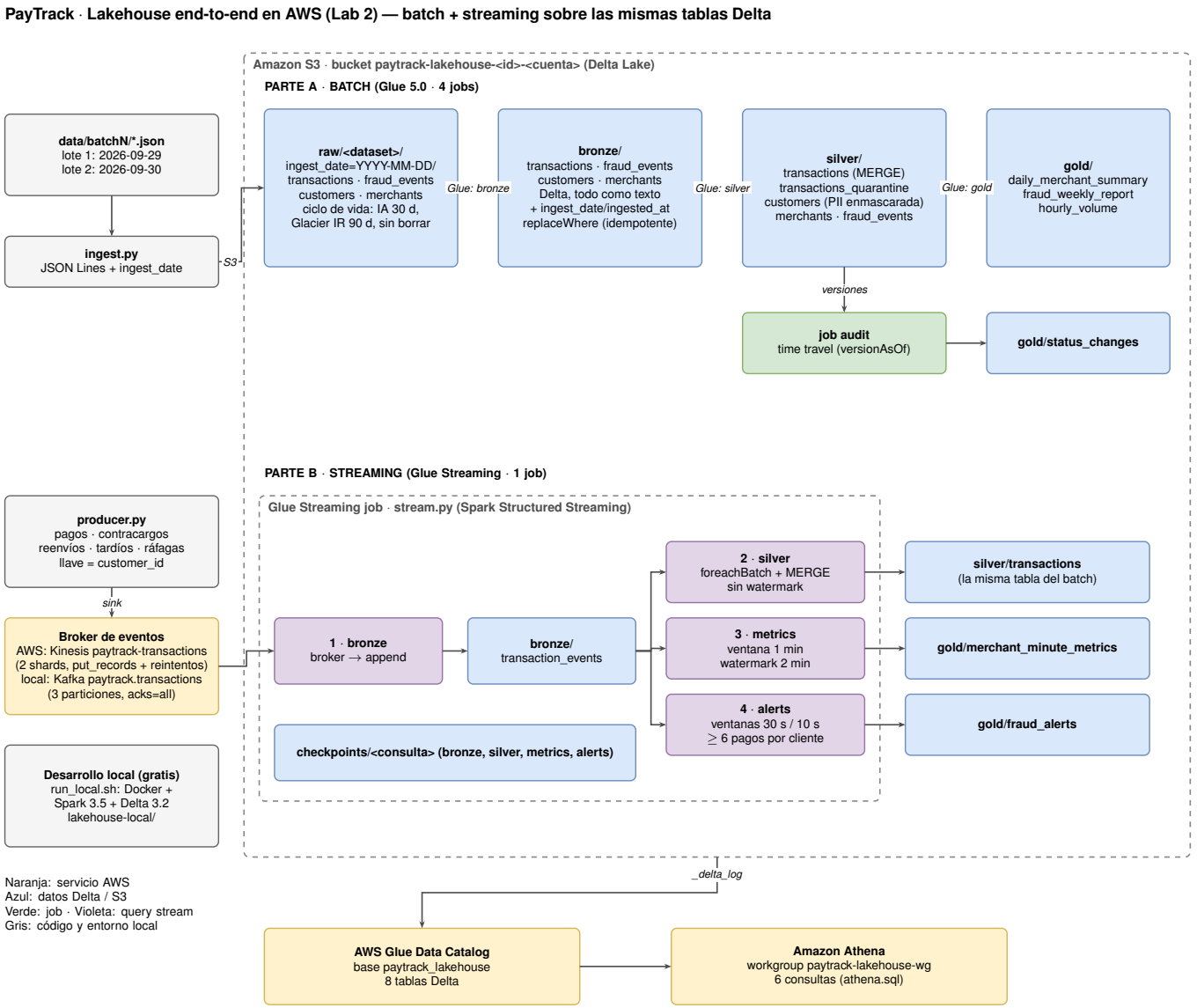
Exp. 3 · Partition key (eventos por partición, 60 s a 20 ev/s):

Llave	Partición 0	Partición 1	Partición 2
`customer_id` (la entregada)	489	344	394
`transaction_id`	429	397	401
constante	0	1 227	0

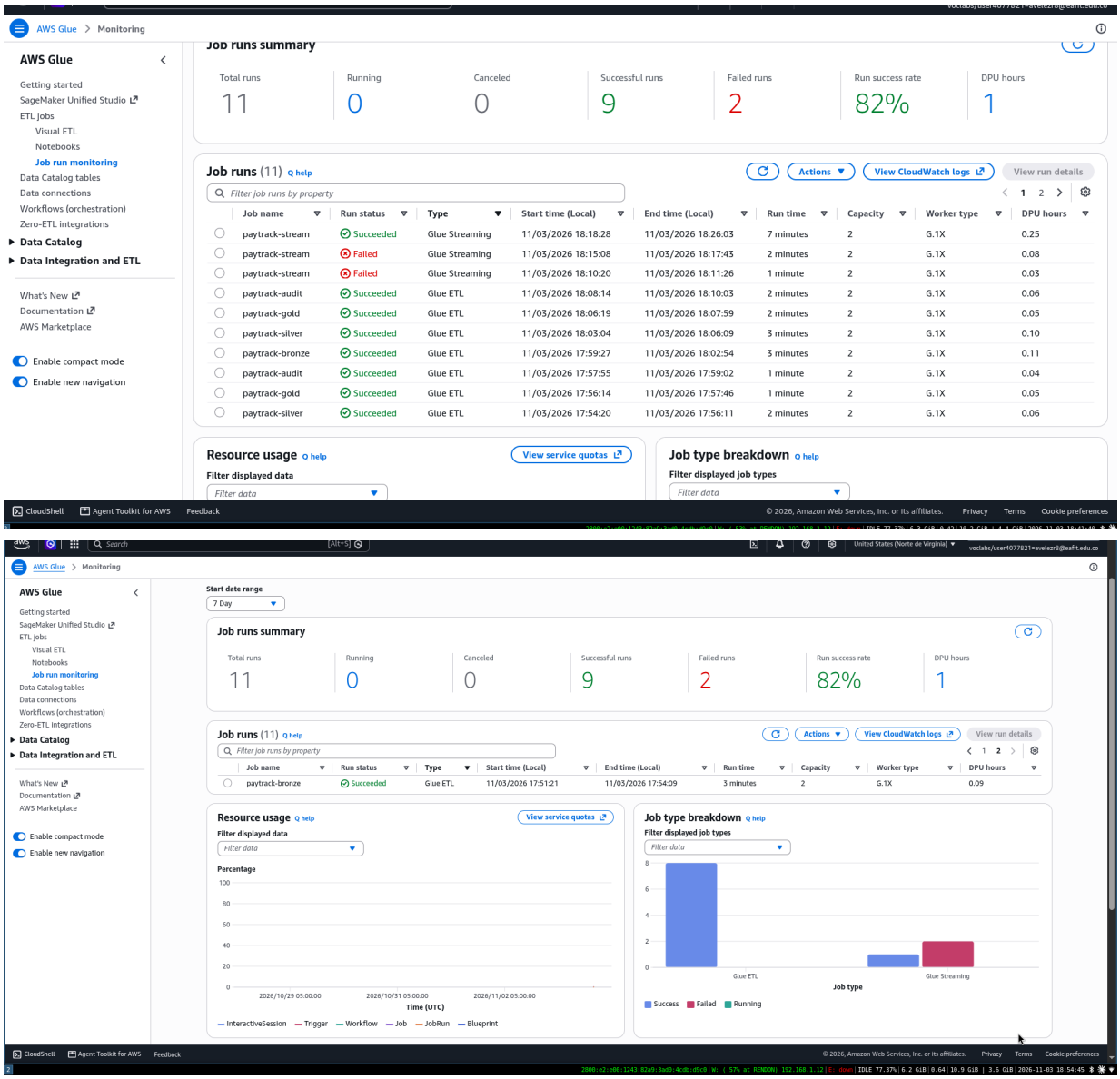
6. Entregables y evidencia

Código en la rama `feat/lab2-lakehouse` (PR al repositorio del curso). En `informe/evidencia/`: capturas de Glue (11 ejecuciones, 9 *Succeeded*, 2 *Failed* previas al arreglo del conector) y de las 6 consultas de Athena, salidas de `run_local.sh` y de `pytest`. Al terminar se ejecutó la limpieza de AWS (bucket vaciado, `serverless remove` y verificación de que no queda ningún stream de Kinesis).

Anexo A · Arquitectura



Anexo B · Evidencia de AWS (Glue)



Job run monitoring de Glue: 11 ejecuciones (4 jobs batch × 2 lotes + 3 del stream: 2 *Failed* previas al arreglo del conector y 1 *Succeeded*). Capturas de las 6 consultas de Athena en `informe/evidencia/`.